

Dsu Complexity

并查集复杂度

本部分内容转载并修改自 [时间复杂度 - 势能分析浅谈](#)，已取得原作者授权同意。

定义

阿克曼函数

这里，先给出 A 的定义。为了给出这个定义，先给出 f 的定义。

定义 f 为：

即阿克曼函数。

这里， $f^k(x)$ 表示将 f 连续应用在 x 上 k 次，即， $f^0(x) = x$ ， $f^k(x) = f(f^{k-1}(x))$ 。

再定义 A 为使得 $f^k(x) \leq y$ 的最小整数值。注意，我们之前将它描述为 $A(x, y)$ ，反正他们的增长速度都很慢，值都不超过 4。

基础定义

每个节点都有一个 rank。这里的 rank 不是节点个数，而是深度。节点的初始 rank 为 0，在合并的时候，如果两个节点的 rank 不同，则将 rank 小的节点合并到 rank 大的节点上，并且不更新大节点的 rank 值。否则，随机将某个节点合并到另外一个节点上，将根节点的 rank 值 +1。这里根节点的 rank 给出了该树的高度。记 x 的 rank 为 $r(x)$ ，类似的，记 x 的父节点为 $p(x)$ 。我们总有 $r(x) < r(p(x))$ 。

为了定义势函数，需要预先定义一个辅助函数 W 。其中， $W(x) = 2^{r(x)}$ 。当 x 是根节点的时候，再定义一个辅助函数 $W(x) = 1$ 。这些函数定义的 W 都满足 $W(x) < W(p(x))$ 且 x 不是某个树的根。

上面那些定义可能让你有点头晕。再理一下，对于一个 x 和 y ，如果 $r(x) < r(y)$ ，总是可以找到一对 z 令 $x = z$ ，而 $y = p(z)$ ，在这个前提下， $W(x) < W(y)$ 。描述了 W 的最大迭代级数，而 A 描述了在最大迭代级数时的最大迭代次数。

对于这两个函数， W 总是随着操作的进行而增加或不变，如果 W 不增加， A 只会增加或不变。并且，它们总是满足以下两个不等式：

考虑 W 、 r 和 A 的定义，这些很容易被证明出来，就留给读者用于熟悉定义了。

定义势能函数 Φ ，其中 Φ 表示一整个并查集，而 $\Phi(x)$ 为并查集中的一个节点。定义 Φ 为：

$\Phi(x) = W(x)$ 或为某棵树的根节点

然后就是通过操作引起的势能变化来证明摊还时间复杂度为 $O(n \log n)$ 啦。注意，这里我们讨论的 find 操作保证了 $r(x)$ 和 $r(y)$ 都是某个树的根，因此不需要额外执行 r 和 p 。

可以发现，势能总是个非负数。另，在开始的时候，并查集的势能为 $\Phi(x) = W(x) = 1$ 。

证明

union(x,y) 操作

其花费的时间为 $\alpha(n)$ ，因此我们考虑其引起的势能的变化。

这里，我们假设 x 被接到 y 上。这样，势能增加的节点仅有 x （从树根变成非树根）， y 的秩可能增加，和操作前 y 的子节点（父节点的秩可能增加）。我们先证明操作前 y 的子节点 z 的势能不可能增加，并且如果减少了，至少减少 1。

设操作前 z 的势能为 ϕ_z ，操作后为 ϕ'_z ，这里 z 可以是任意一个 y 的非根节点，操作可以是任意操作，包括下面的 find 操作。我们分三种情况讨论。

1. x 和 y 并未增加。显然有 $\phi_z \geq \phi'_z$ 。
2. x 增加了， y 并未增加。这里 ϕ_z 至少增加一，即 $\phi_z \geq \phi'_z + 1$ ，势能函数减少了，并且至少减少 1。
3. x 增加了， y 可能减少。但是由于 y 最多减少 1，而 ϕ_z 至少增加 1。由定义，可得 $\phi_z \geq \phi'_z$ 。
4. 其他情况。由于 x 不变， y 不减，所以不存在。

所以，势能增加的节点仅可能是 x 或 y 。而 x 从树根变成了非树根，如果 x 是根，则一直有 $\phi_x \geq \phi'_x$ 。否则，一定有 $\phi_x \geq \phi'_x + 1$ 。即， $\phi_x \geq \phi'_x$ 。

因此，唯一势能可能增加的点就是 y 。而 y 的势能最多增加 1。因此，可得 union 操作均摊后的时间复杂度为 $\alpha(n)$ 。

find(a) 操作

如果查找路径包含 k 个节点，显然其查找的时间复杂度是 $O(k)$ 。如果由于查找操作，没有节点的势能增加，且至少有 $k-1$ 个节点的势能至少减少 1，就可以证明 find 操作的时间复杂度为 $O(\alpha(n))$ 。为了避免混淆，这里用 $\text{find}(a)$ 作为参数，而出现的 find 都是泛指某一个并查集内的结点。

首先证明没有节点的势能增加。很显然，我们在上面证明过所有非根节点的势能不增，而根节点的 ϕ 没有改变，所以没有节点的势能增加。

接下来证明至少有 $k-1$ 个节点的势能至少减少 1。我们上面证明过了，如果 x 或者 y 有改变的话，它们的势能至少减少 1。所以，只需要证明至少有 $k-1$ 个节点的 x 或者 y 有改变即可。

回忆一下非根节点势能的定义， $\phi_z = \text{rank}(z) - \text{rank}(p_z)$ ，而 $\text{rank}(z)$ 和 $\text{rank}(p_z)$ 是使 rank 的最大数。

所以，如果 x 代表 a 所处的树的根节点，只需要证明 $\text{rank}(a)$ 就好了。根据 rank 的定义， $\text{rank}(a) = \text{rank}(p_a)$ 。

注意，我们可能会用 $\text{rank}(a)$ 代表 $\text{rank}(a)$ ，代表 $\text{rank}(a)$ 以避免式子过于冗长。这里，就是 $\text{rank}(a)$ 。

当你看到这的时候，可能会有一种「这啥玩意」的感觉。这意味着你可能需要多看几遍，或者跳过一些内容以后再看。

这里，我们需要一个外接的 rank ，意味着我们可能需要再找一个点 x 。令 x 是搜索路径上在 a 之后的满足 $\text{rank}(x) = \text{rank}(a)$ 的点，这里「搜索路径之后」相当于「是 a 的祖先」。显然，不是每一个 a 都有这样一个 x 。很容易证明，没有这样的 x 的 a 不超过 $\alpha(n)$ 个。因为只有每个 a 的最后一个 x 和 a 以及 p_a 没有这样的 x 。

我们再强调一遍 rank 指的是路径压缩之前 的父节点，路径压缩之后 的父节点一律用 rank 表示。对于每个存在 a 的 x ，总是有 $\text{rank}(x) = \text{rank}(a)$ 。同时，我们有 $\text{rank}(a) \geq \text{rank}(p_a)$ 。由于 $\text{rank}(a) = \text{rank}(p_a)$ ，我们用 rank 来统称，即， $\text{rank}(a) = \text{rank}(p_a)$ 。我们需要造一个 rank 出来，所以我们可以不关注 rank 的值，直接使用弱化版的 rank 。

如果我们将不等式组合起来，神奇的事情就发生了。我们发现， $\text{rank}(a) \geq \text{rank}(p_a)$ 。也就是说，为了从 $\text{rank}(a)$ 迭代到 $\text{rank}(p_a)$ ，至少可以迭代 $\alpha(n)$ 次而不会超过 $\alpha(n)$ 。

显然，有 $\text{rank}(a) \geq \text{rank}(p_a)$ ，且 $\text{rank}(a)$ 在路径压缩时不变。因此，我们可以得到 $\text{rank}(a) \geq \text{rank}(p_a)$ ，也就是说 $\text{rank}(a)$ 的值至少增加 1，如果 $\text{rank}(a)$ 没有增加，一定是 $\text{rank}(a)$ 增加了。

所以， $\text{rank}(a)$ 至少减少了 1。由于这样的 a 节点至少有 $\alpha(n)$ 个，所以最后 $\text{rank}(a)$ 至少减少了 $\alpha(n)$ ，均摊后的时间复杂度即为 $O(\alpha(n))$ 。

为何并查集会被卡

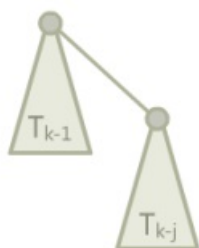
这个问题也就是问，如果我们不按秩合并，会有哪些性质被破坏，导致并查集的时间复杂度不能保证为 $O(n \log n)$ 。

如果我们在合并的时候，较大的合并到了较小的节点上面，我们就将那个较小的节点的 val 设为另一个节点的 val 加一。这样，我们就能保证，从而不会出现类似于满地 `compile error` 一样的性质不符合。

显然，如果这样子的话，我们破坏的就是函数「 y 的势能最多增加」这一句。

存在一个能使路径压缩并查集时间复杂度降至 $O(n)$ 的结构，定义如下：

二项树（实际上和一般的二项树不太一样），其中 j 是常数，为一个 T_{k-j} 加上一个 T_{k-1} 作为根节点的儿子。



边界条件， T_0 都是一个单独的点。

令 val ，这里我们有（证明略）。每轮操作，我们将它接到一个单节点上，然后查询底部的 val 个节点。也就是说，我们接到单节点上的时候，单节点的势能提高了。在 T_{k-1} ， T_{k-j} 的时候，势能增加量为：

变换一下，去掉所有的取整符号，就可以得出，势能增加量， m 次操作就是 $O(m \log n)$ 。

关于启发式合并

由于按秩合并比启发式合并难写，所以很多 dalao 会选择使用启发式合并来写并查集。具体来说，则是对每个根都维护一个 $size$ ，每次将小的合并到大的上面。

所以，启发式合并会不会被卡？

首先，可以从秩参与证明的性质来说明。如果 $size$ 可以代替 $rank$ 的地位，则可以使用启发式合并。快速总结一下，秩参与证明的性质有以下三条：

1. 每次合并，最多有一个节点的秩上升，而且最多上升 1。
2. 总有 $size \geq rank$ 。
3. 节点的秩不减。

关于第二条和第三条，显然满足，然而第一条不满足，如果将 T_{k-1} 合并到 T_{k-j} 上面，则 $rank$ 会增大那么多。

所以，可以考虑使用 $size$ 代替 $rank$ 。

关于第一条性质，由于节点的 $size$ 最多翻倍，所以 $rank$ 最多上升 1。关于第二三条性质，结论较为显然，这里略去证明。

所以说，如果不想写按秩合并，就写启发式合并好了，时间复杂度仍旧是 $O(n \log n)$ 。

本页面贡献者：[orzAtalod](#), [StudyingFather](#), [CCXXI](#), [Enter-tainer](#), [Friendseeker](#), [H-J-Granger](#), [iamtwz](#), [NachtgeistW](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用